

MONGODB INTEGRATION + READ

This video is Part 2 of our Contact App CRUD series. We move from static setup to live database integration! Our primary objectives are:

- Connecting Express.js to MongoDB using **Mongoose**.
- Fetching records using Mongoose queries (`find()` & `findById()`).
- Rendering live data dynamically in EJS templates (`home.ejs` & `show-contact.ejs`).

📌 Architecture Workflow Flowchart



★ 4 ESSENTIAL INTEGRATION STEPS:

1. **1. Install Mongoose:** Express driver.

```
npm install mongoose
```

Copy

2. **2. Include Mongoose:** Import module.

```
const mongoose = require("mongoose");
```

Copy

3. **3. Connect to Database:** Establish connection.

```
mongoose.connect("mongodb://localhost:27017/contact_crud");
```

Copy

4. **4. Create Model with Schema:** Bind schemas.

```
mongoose.model("Contact", contactSchema);
```

Copy

MONGOOSE TYPES + HELPER METHODS

SUPPORTED MONGOOSE DATA TYPES:

- String
- Number
- Date
- Buffer
- Boolean
- Mixed
- ObjectId
- Array
- Decimal128
- Map / UUID
- Double
- Int32

MONGODB COMMANDS VS MONGOOSE SHORTCUTS:

CATEGORY	STANDARD MONGO COMMAND	MONGOOSE HELPER SHORTCUT
Create	<code>insertOne()</code> , <code>insertMany()</code>	<code>create()</code>
Read	<code>find()</code> , <code>findOne()</code>	<code>findById()</code>
Update	<code>updateOne()</code> , <code>updateMany()</code>	<code>findByIdAndUpdate()</code>
Delete	<code>deleteOne()</code> , <code>deleteMany()</code>	<code>findByIdAndDelete()</code>

💡 Why Use ID Helpers?

`findById()` directly queries by MongoDB's unique `_id` field, simplifying syntax compared to `findOne({ _id: id })`!

DB CONNECTION + SCHEMA SETUP

Terminal Installation Workflow:

Stop running Nodemon (Ctrl + C), clear terminal (Ctrl + L), install Mongoose, and restart server:

```
npm install mongoose
npm run start
```

[Copy](#)

Database Connection in index.js:

```
const mongoose = require("mongoose");

// Database Connection
mongoose
  .connect("mongodb://localhost:27017/contact_crud")
  .then(() => {
    console.log("Database Connected");
  });
```

[Copy](#)

Modular Folder Setup:

Create folder models/ and file contact.model.js inside it.

Defining contactSchema inside models/contact.model.js:

```
const mongoose = require("mongoose");

const contactSchema = mongoose.Schema({
  firstName: { type: String },
  lastName: { type: String },
  email: { type: String },
  phone: { type: String },
  address: { type: String }
});
```

[Copy](#)

MONGOOSE MODEL + SAMPLE DATA

1. CREATING + EXPORTING MODEL

Inside models/contact.model.js, compile schema into Model and export:

```
const Contact = mongoose.model("Contact", contactSchema);  
module.exports = Contact;
```

[Copy](#)

2. IMPORTING MODEL IN INDEX.JS

```
const Contact = require("../models/contact.model");
```

[Copy](#)

3. VERIFYING DATABASE IN MONGO SHELL

```
show dbs           // Shows contact_crud  
use contact_crud   // Switched to db contact_crud  
show collections   // Shows contact collection
```

[Copy](#)

4. INSERTING SAMPLE DATA IN MONGO SHELL

```
db.contact.insertOne({  
  firstName: "Yahu",  
  lastName: "Baba",  
  email: "yahu@example.com",  
  phone: "9876543210",  
  address: "Delhi"  
});  
  
db.contact.find();
```

[Copy](#)

READING ALL RECORDS (HOME ROUTE)

Asynchronous Database Querying:

Database queries take time to complete. Therefore, the route handler *MUST* use `async/await` keywords!

 Home Route Handler in `index.js`:

[Copy](#)

```
app.get("/", async (req, res) => {  
  const contacts = await Contact.find();  
  
  // Step A: Test with JSON output first  
  // res.json(contacts);  
  
  // Step B: Render home.ejs with contacts data  
  res.render("home", {  
    contacts  
  });  
});
```

Object Shorthand Note:

Writing `{ contacts }` is equivalent to `{ contacts: contacts }` in ES6 JavaScript when key name matches variable name!

DYNAMIC EJS TABLE LOOP

Inside `views/home.ejs`, remove hardcoded dummy rows inside `<tbody>` and loop through array received from Express:

```
<% contacts.forEach((contact) => { %>
  <tr>
    <td><%= contact.firstName %></td>
    <td><%= contact.lastName %></td>
    <td><%= contact.email %></td>
    <td><%= contact.phone %></td>
    <td>
      <a href="/show-contact/<%= contact._id %>">View</a>
      <a href="/update-contact/<%= contact._id %>">Edit</a>
      <a href="/delete-contact/<%= contact._id %>">Delete</a>
    </td>
  </tr>
<% }) %>
```

[Copy](#)

Appending MongoDB `_id` to Action Links:

Each contact row passes its unique database identifier `<%= contact._id %>` directly in anchor tags so Express can target that specific document!

READING SINGLE CONTACT

1. PARAMETRIC ROUTE + FINDBYID() QUERY

Update routes in index.js to expect an :id URL parameter:

```
// View Single Contact Route
app.get("/show-contact/:id", async (req, res) => {
  const contact = await Contact.findById(req.params.id);
  res.render("show-contact", {
    contact
  });
});
```

[Copy](#)

2. DISPLAYING SINGLE OBJECT IN SHOW-CONTACT.EJS

```
<p>First Name: <%= contact.firstName %></p>
<p>Last Name: <%= contact.lastName %></p>
<p>Email: <%= contact.email %></p>
<p>Phone: <%= contact.phone %></p>
<p>Address: <%= contact.address %></p>
<a href="/update-contact/<%= contact._id %>">Edit</a>
<a href="/delete-contact/<%= contact._id %>">Delete</a>
<a href="/">Cancel</a>
```

[Copy](#)

VIDEO #10 SUMMARY CHECKLIST

- ☐ **Mongoose Connect:** Configured `mongoose.connect("mongodb://localhost:27017/contact_crud")`.
- ☐ **Model File:** Created `models/contact.model.js` schema & exported `Contact` model.
- ☐ **Async/Await Query:** Used `await Contact.find()` inside async handler.
- ☐ **EJS Loop:** Rendered records with `contacts.forEach()` in `home.ejs`.
- ☐ **findById():** Queried single document using `Contact.findById(req.params.id)`.

Next Topic Sneak-Peek

In Video #11, we will process HTML Form submissions to Create, Update, and Delete database records in MongoDB!